

Welcome to Computer Science

From Using Technology → Understanding Technology

These notes summarize what we covered in class. Use them to revise, not to learn the topic for the first time — the real learning happened in the room, with the discussion and the live drawings. Keep this handy; we will refer back to several of these diagrams throughout the program.

In this handout

1. Welcome to Computer Science
 2. How Computers Solve Problems
 3. From Problem to Program
 4. Algorithms
 5. Think Like a Programmer
 6. Why Computers Need Precise Instructions
 7. From Programming to AI
 8. Your Learning Journey
- Try It Yourself · Remember These · Vocabulary
- Homework · Quick Answers / Hints

1

Welcome to Computer Science

You use technology every day — a phone, an app, a game, a chat message. Today's class asks one question: **what is actually happening underneath all of that?**

What is Computer Science?

Computer Science is the study of computation, information, algorithms, and problem solving.

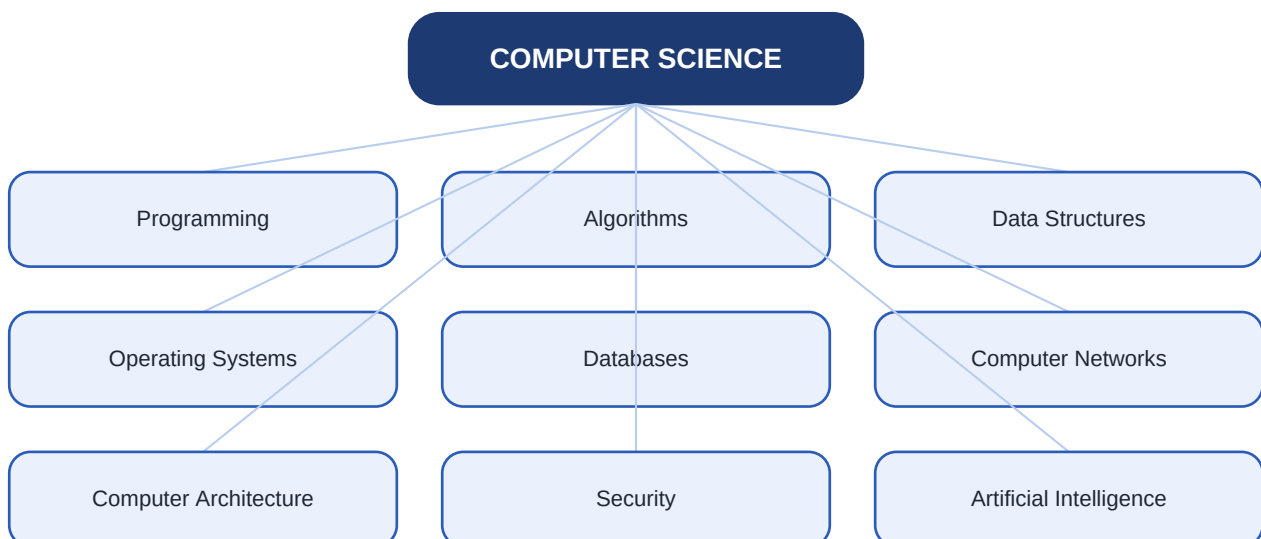
You don't need to memorize that sentence. What matters is the idea: CS is about how problems get solved using computers — not just about writing code.

Computer Science ≠ Coding

Coding (writing programs) is **one part** of Computer Science — an important one, but only one room in a much bigger building. Computer Science also covers how computers store information, how they talk to each other over a network, how they stay secure, and more.

A map of Computer Science

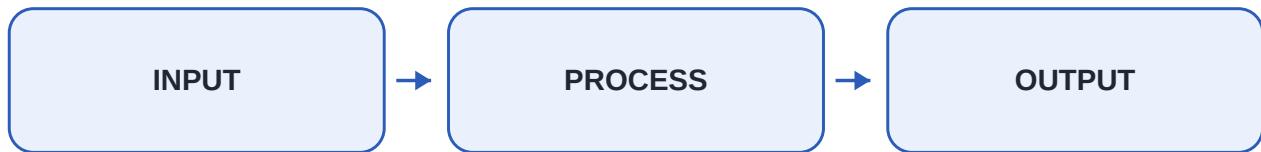
Here are some of the major areas inside Computer Science. You don't need to know what each one means yet — this is just the map. We will visit these areas gradually over the next four months.



2

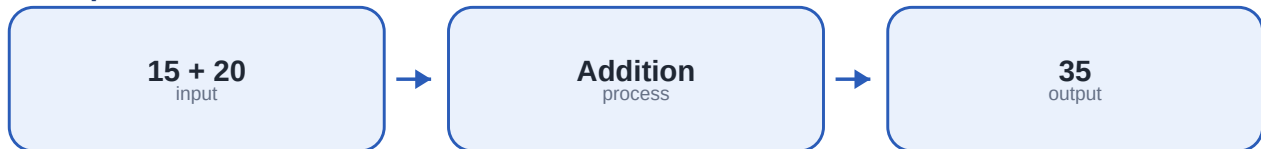
How Computers Solve Problems

Almost anything a computer does can be described with one simple pattern:



Term	Meaning
INPUT	The information you give the computer.
PROCESS	What the computer does with that information.
OUTPUT	The result you get back.

Example — a calculator



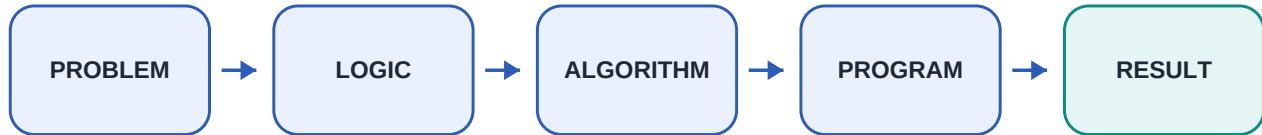
You give the calculator two numbers and an operation (input), it performs the addition (process), and it shows you the answer (output).

Note: This is a simple mental model, and it's genuinely useful for understanding a huge range of systems. But real-world software can be far more complex than three boxes — this is a starting point for thinking, not a rule that every system follows exactly.

3

From Problem to Program

This is one of the most important ideas in the whole program — everything we build over the next four months fits somewhere on this chain.



Step	What it means
Problem	Something in the real world that needs solving.
Logic	Your thinking about how to solve it — before writing anything down.
Algorithm	Your thinking written down as clear, ordered steps.
Program	Those steps translated into a programming language a computer can run.
Result	What the computer produces when it runs the program.

Keep this chain in your notes — we will come back to it constantly.

4

Algorithms

What is an algorithm?

An algorithm is a finite sequence of clear steps used to solve a problem.

Example — find the larger of two numbers

A = 15 B = 21

1. Take two numbers.
2. Compare them.
3. If A is greater than B, A is the larger number.
4. Otherwise, B is the larger number.

That's it — a complete algorithm. Notice it doesn't mention any programming language at all.

Algorithm ≠ Program

	Algorithm	Program
What it is	A solution strategy (the “recipe”)	The strategy written in a programming language (the “dish”)
Depends on a language?	No — works everywhere	Yes — Python and C code look different
Who can follow it?	A human or a computer	Only a computer runs it directly

The same algorithm can become a program in Python, in C, or in any other language — the steps don't change, only the language does.

5

Think Like a Programmer

Before you can write instructions for a computer, it helps to think a certain way. This is called **computational thinking**, and it has four parts.

DECOMPOSITION

Breaking a big problem into smaller pieces — e.g. planning a birthday party: invitations, food, venue, guest list.

PATTERN RECOGNITION

Noticing two problems are similar underneath — e.g. comparing two numbers and comparing two heights use the same idea.

ABSTRACTION

Ignoring details that don't matter right now — e.g. driving a car, you just think "accelerator = go."

ALGORITHMIC THINKING

Expressing a solution as clear, ordered steps — exactly what we did with "larger of two numbers."

You already used all four of these today without realizing it.

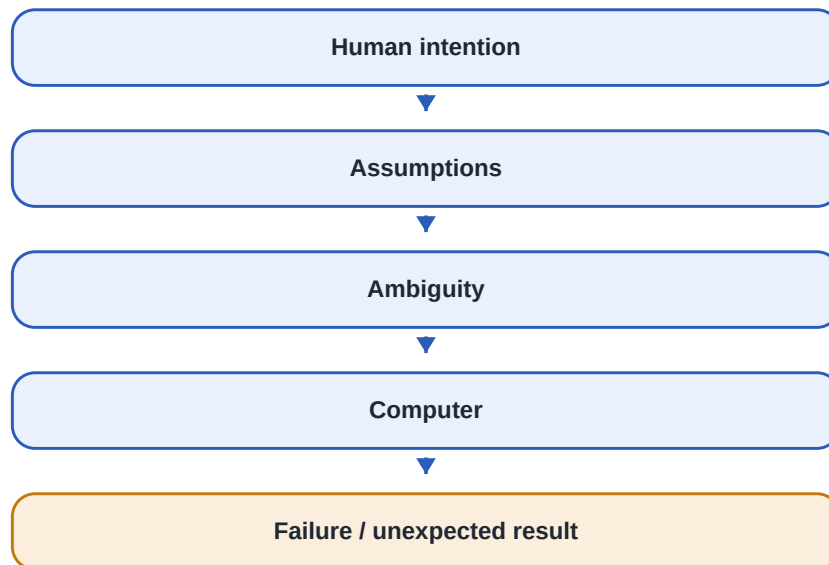
6

Why Computers Need Precise Instructions

In class, we tried an activity: “**Teach a computer how to make tea.**” One of you gave instructions, step by step, while the instructor acted as a computer who does exactly what it's told — nothing more, nothing assumed.

It didn't go smoothly. Instructions like “boil the water” led to more questions: *Where is the water? How much? Where is the pan? What if there's no gas available?*

That gap is the whole point:



Humans naturally fill in missing details using common sense. Computers don't — they only do exactly what they are told, in exactly the order they are told it.

Programming requires us to turn human intentions into precise instructions.

This is why “give the computer clear steps” (an algorithm) matters so much before you ever start writing a program.

7

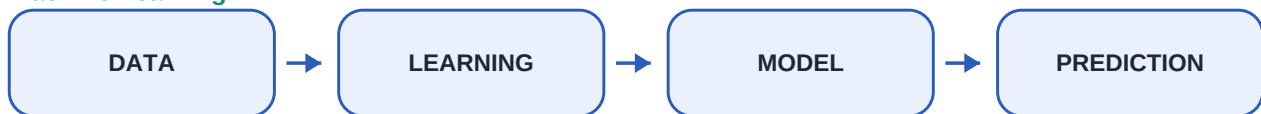
From Programming to AI

Programming, Computer Science, and AI are not separate worlds — AI is built on top of the same foundations we started today. Here's a simple, conceptual comparison of three approaches to building systems (we are not learning how any of these actually work yet):

Traditional Programming



Machine Learning

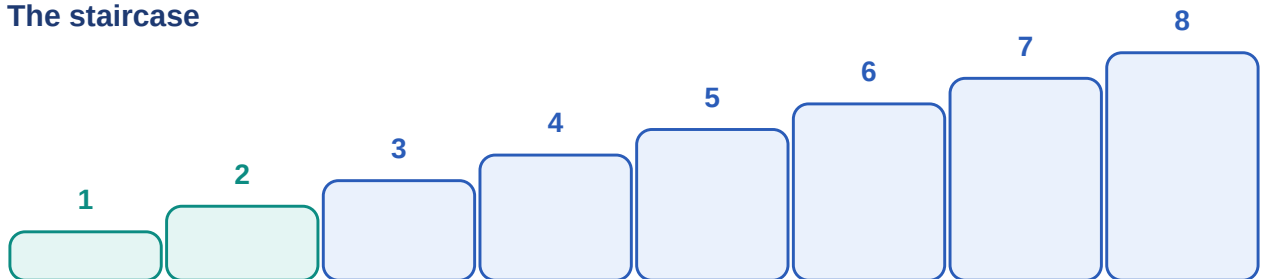


Generative AI



- In **traditional programming**, a human writes the rules.
- In **machine learning**, the rules are learned from data instead of written by hand.
- In **generative AI**, a model generates new content — text, images, and more — in response to a prompt.

The staircase



1. Programming → 2. Computer Science → 3. Mathematics → 4. Data → 5. Machine Learning → 6. Deep Learning → 7. LLMs → 8. Agents

We are not jumping straight to AI. We are building the staircase, one step at a time — starting with programming and Computer Science, right now.

8

Your Learning Journey

The four-month program is built in the same spirit as today's staircase — each month exists because the one before it makes it possible.

Month	Focus	Why it comes here
Month 1	Foundations — computing, Python, programming logic, basic math	You need to know how to think and give instructions before anything else makes sense.
Month 2	CS & Problem Solving — C, DSA, algorithms, SQL, discrete math	Once you can program, you learn how to solve problems well and understand how computers really work.
Month 3	Engineering + AI — Linux, Git/GitHub, APIs, ML, Generative AI, Agents, Math for AI	With solid fundamentals, you're ready to connect real software engineering practice with AI.
Month 4	Consolidation — revision, projects, assessments, Demo Day	You bring everything together and show what you built.

We won't rush to the “exciting” AI topics before the foundations are solid — that's how the staircase stays standing.



Try It Yourself

Answer these in your own words. Don't worry about getting them “perfectly” right — the goal is to think it through.

1. Give one example of an input, a process, and an output from your own day.
2. Explain what an algorithm is, in your own words.
3. What is the difference between an algorithm and a program?
4. Give one example of decomposition from everyday life.
5. Write 5 steps for a simple everyday activity of your choice.

Your answers

1.

2.

3.

4.

5.



Remember These

- Computer Science is broader than coding — coding is one part of it.
- A program is a sequence of instructions a computer can execute.
- Input → Process → Output is a simple, useful way to think about how systems work.
- An algorithm is a solution strategy; a program is that strategy written in a programming language — they are not the same thing.
- Computers need precise instructions because they don't fill in missing assumptions the way humans do.
- Computational thinking (decomposition, pattern recognition, abstraction, algorithmic thinking) helps us solve problems before we ever write code.
- Programming and Computer Science are the foundation that AI is built on.
- We build the staircase one step at a time — we don't jump to the top.



Vocabulary

Term	Meaning
Computer Science	The study of computation, information, algorithms, and problem solving.
Program	A sequence of instructions that a computer can execute.
Programming	Giving a computer precise instructions to solve a problem.
Algorithm	A finite sequence of clear steps used to solve a problem.
Input	Information given to a system before it does anything.
Process	What a system does with the input it receives.
Output	The result a system produces.
Application	Software designed to perform a useful task for users.
Abstraction	Ignoring details that don't matter, and focusing on what does.
Decomposition	Breaking a big problem into smaller, easier pieces.
Machine Learning	An approach where a system learns rules from data instead of a human writing them by hand.
Generative AI	AI that generates new content — such as text or images — in response to a prompt.
LLM	Large Language Model — an AI model that processes and generates human language.



Homework — Assignment 1

Think Like a Programmer

Pick **one** everyday process from the list below (or propose your own):

- Ordering food
- Online shopping
- Sending a message
- Booking a cab
- ATM withdrawal
- Booking a train ticket
- Making tea
- Going to college

For your chosen process:

1. Identify its **Input**, **Process**, and **Output**.
2. Write a simple **algorithm** with **at least 5 numbered steps**.
3. Use plain language — no code, no programming syntax required.

Format

- Keep it to about one page (handwritten photo, doc, or text file).
- Clearly label Input, Process, and Output as three separate items.
- Number each step of your algorithm.

Tip: Think back to the tea activity from class — that level of detail (clear, no missing assumptions) is what we're looking for.

If 5 steps feels difficult, attempt it anyway — a good-faith attempt with the Input/Process/Output split matters more than a perfect answer.

Want a challenge? Add a decision point to your algorithm — a step where what happens next depends on a condition (for example: “if the balance is insufficient, the withdrawal fails”). You don't need to know any special syntax for this — just describe it in plain English.



Quick Answers / Hints

Use these only after you've tried the questions yourself.

- 1 **Input/Process/Output example** — Think of something you did this morning. For example: an alarm sound (input) → waking up and getting ready (process) → leaving for college (output). See Section 2.
- 2 **What is an algorithm** — Think “a recipe made of clear steps,” like the “larger of two numbers” example in Section 4.
- 3 **Algorithm vs. program** — The algorithm is the recipe; the program is the dish made in one specific kitchen (language). See the comparison table in Section 4.
- 4 **Decomposition example** — Pick a big task (like preparing for an exam) and break it into smaller parts (revise notes, solve practice questions, review mistakes). See Section 5.
- 5 **5-step algorithm** — Pick something you do daily and write it out the way we built the tea algorithm in class — one clear action per step, in order. See Section 6.